

Smart ETA Prediction Project Report

Smart ETA Prediction is a production-track ETA prediction system for a taxi dispatch context, using the NYC TLC public benchmark dataset as a prototype while keeping the architecture compatible with future company data.

1. Project Structure

- `app.py` — Production FastAPI inference service with fallback routing and healthy service metadata.
- `app_streamlit.py` — Streamlit dashboard and demo interface for running API requests and monitoring model metrics.
- `simulate_stream.py` — Synthetic ride stream generator and latency/load test driver.
- `src/` — Shared source of truth: data preparation, feature engineering, model training, evaluation and utility code.
- `data/raw`, `data/interim`, `data/processed` — Raw data, intermediate cleaned data, and final model-ready processed files.
- `configs/` — YAML configuration files for paths, metadata, model parameters and split governance.
- `notebooks/` — Numbered notebooks (01 to 08) mapped to pipeline stages.
- `models/` — Model bundles, serialized champion model and benchmark comparison artifacts.
- `tests/` — Automated pytest suite for API and source-level validation.

2. Pipeline Steps

- Step 1: Load and clean the NYC TLC dataset into the interim layer with schema and bounds validation.
- Step 2: Build chronological train/validation/test Parquet splits using no shuffling, preventing temporal leakage.
- Step 3: Feature engineering creates haversine and Manhattan distance, temporal features, rush-hour flags, and cyclical hour signals.
- Step 4: Train and benchmark baseline, Random Forest, XGBoost, LightGBM, and CatBoost regressors.
- Step 5: Assess metrics, business thresholds, and generated slices for short trips, long trips, rush hour and off-peak trips.
- Step 6: Persist champion model and comparison metrics as model artifacts.
- Step 7: Validate API health, model-info, and predictions through FastAPI endpoints.

3. Tech Stack

- Python ≥ 3.10 with pandas, numpy, pyarrow, pyyaml, scikit-learn, joblib, matplotlib, seaborn, jupyter and pytest.
- Model stack: xgboost, lightgbm, catboost, RandomForestRegressor and a rule-based average-speed baseline.
- Deployment stack: FastAPI, Uvicorn, Pydantic, Streamlit, HTTP client libraries for API simulation and requests.
- Project governance: config-driven paths and rules in YAML so data swapping is controlled by config rather than hardcoded values.

4. Commands and Actions

- Environment creation: `python -m venv .venv` and source `.venv/bin/activate` or `.venv\Scripts\activate`.
- Dependency install: `pip install -r requirements.txt`.
- Pipeline invocation: `python scripts/run_pipeline.py`.
- API launch: `uvicorn app:app --host 0.0.0.0 --port 8000 --reload`.
- Streamlit dashboard: `streamlit run app_streamlit.py --server.port=8501`.
- Streaming simulation: `python simulate_stream.py --num-trips 15`.
- Tests: `pytest tests/ -v`.
- Static compile check: `python -m py_compile $(find src -name "*.py")` from the Makefile pattern.

5. Notebooks and Data Mapping

- 01_dataset_audit.ipynb — Data audit and feasibility check.
- 02_data_cleaning.ipynb — Cleaning rules and interim validation.
- 03_dataset_design.ipynb — Train/val/test split and dataset design.
- 04_feature_engineering.ipynb — Feature generation and distance/time features.
- 05_eda.ipynb — Exploratory data analysis and feature distributions.
- 06_baseline_modeling.ipynb — Historical average speed baseline model.
- 07_advanced_modeling.ipynb — Model benchmarking for RF, XGBoost, LightGBM and CatBoost.
- 08_evaluation.ipynb — Final evaluation, metrics and business gates.

6. Quality and Business Gates

- The evaluation layer expresses MAE in minutes and seconds, RMSE in minutes, MAPE in percentage and R^2 score.
- Business thresholds from `metrics.py` compare $MAE \leq 3.5$ minutes and $R^2 \geq 0.60$ for the champion model.
- The API returns quality flags for HIGH_CONFIDENCE, ESTIMATED, FALLBACK_ROUTING and DEGRADED.